

Agentes de programação implementando uma API de encurtar URLs

Relatório metodológico do piloto · 24/09/2026 · Rafael Moura

1. Resumo

Quatro configurações de agente de programação (modelo + harness) receberam o mesmo enunciado: implementar, sozinhas e em um container isolado, uma API HTTP de encurtar URLs com stack livre. Cada entrega foi congelada e verificada por um avaliador automatizado em um ambiente limpo, contra 35 requisitos do contrato, e teve o tempo de resposta medido sob carga.

- **As quatro configurações tiveram a entrega aceita, com 35 de 35 requisitos:** Opus (Claude Code), Sol e Astra (Codex) e Muse (OpenCode).
- **Tempo de trabalho:** de 197 s (Opus) a 602 s (Muse), no máximo um terço do prazo adotado para a coleta (1.800 s).
- **Custo estimado pelo preço público de API:** de US\$ 0,28 (Sol) a US\$ 0,95 (Astra) por tentativa. Nenhuma foi paga por token.
- **Stack:** três escolheram Python com a biblioteca padrão e SQLite; o Opus escolheu Go, com um log em arquivo. Nenhuma entrega usou dependência externa.
- **Desempenho da API:** o Opus responde em cerca de 0,05 ms e atende de 110 mil a 190 mil requisições por segundo; as entregas em Python ficam entre 1,3 e 1,6 ms e 740 a 1.440 req/s. Detalhes de implementação dentro da mesma stack mudaram a latência do POST em 26 vezes.
- **É um piloto:** uma tentativa por configuração, resultados não oficiais. O objetivo principal foi validar o método e o instrumento: o piloto encontrou cinco problemas deles, quatro corrigidos e um documentado.

2. Objetivo e perguntas (GQM)

O estudo segue o método GQM (Goal–Question–Metric). O objetivo é analisar o processo de desenvolvimento e as entregas de configurações completas de agentes (modelo, harness e skills) na implementação de uma API, para caracterizá-las e compará-las de forma exploratória quanto a correção funcional, robustez, manutenibilidade, eficiência e desempenho da API entregue, do ponto de vista de quem precisa revisar e usar as entregas.

O objeto comparado é a configuração completa: o desenho não isola o efeito do modelo, do harness ou das skills.

Pergunta	Métricas principais	O que o piloto mostrou
Q1. A entrega é aceitável pelo contrato?	M1 aceitação (A _i); M2 frequência	4 de 4 aceitas
Q2. Quais requisitos são atendidos?	M3 = S/N; M4 matriz por requisito	35 S em todas; nenhum V ou U
Q3. Robustez em erros e limites	M5 requisitos de robustez	RN01–RN15 e RNF06–RNF09 atendidos
Q4. Executa em ambiente limpo?	M6 build e início; M7 intervenção	Todas sem intervenção
Q5. Que recursos foram necessários?	M8 tempo; M9 custo; M10 intervenção humana; M14 tokens	197–602 s; US\$ 0,28–0,95; nenhuma intervenção
Q6. Quanto varia entre tentativas?	M11 dispersão	Sem dados: uma tentativa por configuração
Q7. Qualidade interna	M12 rubrica qualitativa	Não avaliada no piloto
Q8. Que stack foi escolhida?	M15 perfil da stack	3× Python + SQLite; 1× Go
Q9. Com que tempo a API responde?	M16 latência sob carga fixa	0,04 ms (Go) a 42 ms (POST do Sol)

A aceitação é binária: A_i = 1 somente se os 35 requisitos obrigatórios tiverem resultado S (satisfeito). Cada requisito recebe S, V (violado, com evidência) ou U (sem resultado conclusivo, por exemplo quando uma pré-condição falha). Tempo, tokens, custo, stack e latência não entram na aceitação: são medidas para comparar as configurações.

3. Tarefa e contrato

A tarefa é uma API de encurtar URLs com cinco operações: GET /health, POST /api/links (criar, com alias e expiração opcionais), GET /{code} (redirecionar com 302 e contar a visita), GET /api/links/{code} (consultar) e DELETE /api/links/{code}. O contrato fixa formatos, códigos de erro (400, 404, 409, 410, 422) e o formato do corpo de erro.

A entrega deve ter build.sh e start.sh, escutar na porta 8080, persistir somente em DATA_DIR, usar BASE_URL nas URLs curtas, funcionar sem rede em execução, preservar os dados após SIGTERM e não perder visitas, códigos ou aliases sob requisições simultâneas.

O contrato tem duas partes. **A Parte A (enunciado) é o único material enviado ao agente.** A Parte B (catálogo de 35 requisitos obrigatórios e os casos de verificação reservados) nunca sai do computador, para que o agente não possa otimizar contra os testes. Os 35 requisitos se dividem em 11 funcionais (RF), 15 regras de validação e erro (RN) e 9 não funcionais (RNF).

4. Participantes

Configuração	Modelo e esforço	Harness	Observação
Opus	claude-opus-5-5, high	Claude Code 2.1.281	Plano Pro do Claude
Sol	gpt-6-sol, high	Codex 0.156.1	Assinatura ChatGPT
Astra	gpt-6-astra, high	Codex 0.156.1	Mesma conta e harness do Sol; incluído após o piloto
Muse	muse-spark-1.3-contributor-free, xhigh	OpenCode 1.18.32	Rota gratuita; dados podem ser usados para treino

O MiMo-V2.6-Flash Free (OpenCode) foi testado e retirado: nas duas tentativas, o modelo esgotou o limite de 32 mil tokens de saída raciocinando em um único passo, antes de escrever qualquer arquivo. O Gemini já tinha sido descartado no piloto de infraestrutura. Nenhuma configuração usa skills.

5. Ambiente e stack disponível

Uma única imagem Docker serve para o agente e para o avaliador (Debian bookworm, fixada por digest). As toolchains têm versão e checksum fixos no Dockerfile, e os binários dos harnesses são fixados por hash em um manifesto versionado.

Componente	Versão disponível
Python	3.11, com pip e venv; SQLite 3.40
Node.js	24 LTS, com npm
Go	1.27 (GOTOOLCHAIN=local)
Java	25 (Temurin JDK), com Maven 3.9
Compilação nativa	gcc e make (build-essential)
Registros de pacotes	PyPI, npm, proxy de módulos Go e Maven Central — só durante desenvolvimento e build

A stack é livre dentro desse ambiente. Em execução, a entrega não tem acesso a rede externa. Nenhuma das quatro entregas usou dependências: todas ficaram na biblioteca padrão da linguagem escolhida.

6. Isolamento: como evitamos acessos indevidos

O agente roda inteiro dentro de um container novo, criado para cada tentativa e destruído ao final. As barreiras se sobrepõem, e cada tentativa verifica automaticamente as principais antes de começar.

Container e sistema de arquivos

- Raiz do sistema somente leitura, usuário não root (UID 1001), todas as capabilities removidas, no-new-privileges e processo init.
- Home, /tmp e /workspace em tmpfs, descartados ao final; o runner verifica que home e workspace começam vazios.
- Nada do host é montado: nem o repositório, nem entregas anteriores, nem histórico, nem o socket do Docker. O runner verifica a ausência do home do host e do socket.
- Limites fixos de recursos: 4 GiB de memória, 2 CPUs e 512 processos.

Rede

- O container fica em uma rede Docker interna, sem rota para fora. A única saída é um proxy que aceita apenas CONNECT na porta 443 para uma lista explícita: o provedor do próprio modelo e os seis registros de pacotes.
- Antes de cada tentativa, o runner confirma que o PyPI é alcançável, que um domínio fora da lista (example.com) é bloqueado pelo proxy e que a conexão direta a um IP externo falha. Se algo não confere, a tentativa não começa.
- Nenhuma porta é publicada no host. O proxy registra os destinos pedidos (ALLOW/DENY), sem conteúdo, e esse log faz parte das evidências.
- No piloto, nenhum agente tentou um destino bloqueado. Os únicos bloqueios foram a verificação do próprio runner e endereços de telemetria do Codex.

Credenciais e material

- Cada agente recebe só a credencial do próprio provedor, por stdin, gravada com permissão 0600 no tmpfs. Muse não recebe credencial.
- O prompt é o preâmbulo mais a Parte A do contrato. O runner recusa enviar o texto se encontrar marcas da Parte B ou valores não definidos, e registra o SHA-256 do contrato, da Parte A e do prompt.
- As saídas do agente são gravadas com as credenciais redigidas, e a entrega congelada é verificada para garantir que não contém nenhum valor de credencial.

Ferramentas do agente

- Sem busca na web, sem subagentes, sem skills e sem servidores MCP. Claude Code: Bash, Read, Write, Edit, Glob e Grep. OpenCode: bash, leitura, edição, busca e lista de tarefas. Codex: execução dentro do container.
- Sem interação humana durante a tentativa: o prompt avisa que ninguém responderá perguntas.

Avaliação separada da produção

- A entrega é congelada (tar com hash da árvore) e o container do agente é destruído antes da avaliação.

- O avaliador roda em containers novos: o build tem rede só para os registros; o servidor roda sem rede, e três sondas confirmam que não há rota externa. Os checks rodam de um container cliente na mesma rede interna.

O que não foi certificado

Resolução DNS de serviços do host e tentativas deliberadamente adversariais de escapar do container não foram testadas. As medidas acima cobrem um agente que segue o enunciado, não um atacante.

7. Execução e avaliação

Runner

Cada tentativa segue o mesmo fluxo: prompt e rede → container do agente → provisionamento e verificações → tentativa com prazo (SIGTERM e depois SIGKILL) → encerramento dos processos → congelamento da entrega → limpeza → avaliação. O resumo de cada tentativa registra configuração e hashes, tempos, tokens informados pelo harness, recursos do container (memória, CPU, disco), chamadas de ferramenta, como o harness terminou e os destinos vistos pelo proxy.

Avaliador

- Executa build.sh (sem prazo; teto operacional de 1 hora) e start.sh, e espera GET /health responder 200 em até 60 s.
- Roda os checks funcionais, de validação, de erro e de concorrência (50 redirecionamentos, 20 criações com o mesmo alias e 50 criações simultâneas).
- Durabilidade: SIGTERM e novo início com o mesmo DATA_DIR; depois, restaura todo o resto ao estado pós-build e reinicia de novo, para provar que o estado está só em DATA_DIR.
- Um defeito do avaliador nunca vira V: a avaliação fica inconclusiva.

O avaliador foi validado antes do piloto com uma implementação de referência (35 S) e 26 controles: 3 positivos, que devem passar, e 23 versões da referência com defeitos plantados. Todos produziram exatamente os resultados esperados.

Medição de latência (Q9)

Fora da aceitação. A entrega sobe no mesmo ambiente limpo, com DATA_DIR em disco, servidor e gerador de carga em CPUs separadas. Para GET /{code} e POST /api/links: 500 req/s por 30 s (latência contada do instante agendado) e 32 conexões simultâneas por 10 s (capacidade). Só respostas corretas entram nos percentis, e a soma das visitas deve bater com os redirecionamentos. Cada entrega foi medida três vezes; a mediana e a vazão variaram menos de 2% entre as medições.

8. Resultados

	Opus	Sol	Astra	Muse
Requisitos (S/35)	35	35	35	35
Tempo da tentativa	197 s	374 s	367 s	602 s
Chamadas de ferramenta	13	12	10	45
Tokens gerados (saída + raciocínio)	21 mil	17 mil	11 mil	46 mil
Tokens lidos do cache	140 mil	341 mil	167 mil	1,99 milhão
Custo estimado (API)	US\$ 0,65	US\$ 0,28	US\$ 0,95	US\$ 0,63*
Stack	Go, log em arquivo	Python, SQLite	Python, SQLite	Python, SQLite
Código / testes	617 linhas / 10	274 / 3	272 / 14	540 / 0
GET em carga normal (p50)	0,04 ms	1,32 ms	1,51 ms	1,50 ms
POST em carga normal (p50)	0,05 ms	42,3 ms	1,62 ms	1,57 ms
Capacidade de GET	192 mil/s	1.440/s	1.260/s	760/s
GET p99 com 32 conexões	0,56 ms	530 ms	430 ms	47 ms

* Preço do Muse Spark 1.3 pago; a rota usada foi a gratuita. Custos pelo preço público de API consultado em 24/09/2026 (Anthropic, OpenAI, OpenCode Zen); nenhuma tentativa foi paga por token. A estimativa do Opus confere com o custo informado pelo próprio Claude Code (US\$ 0,655).

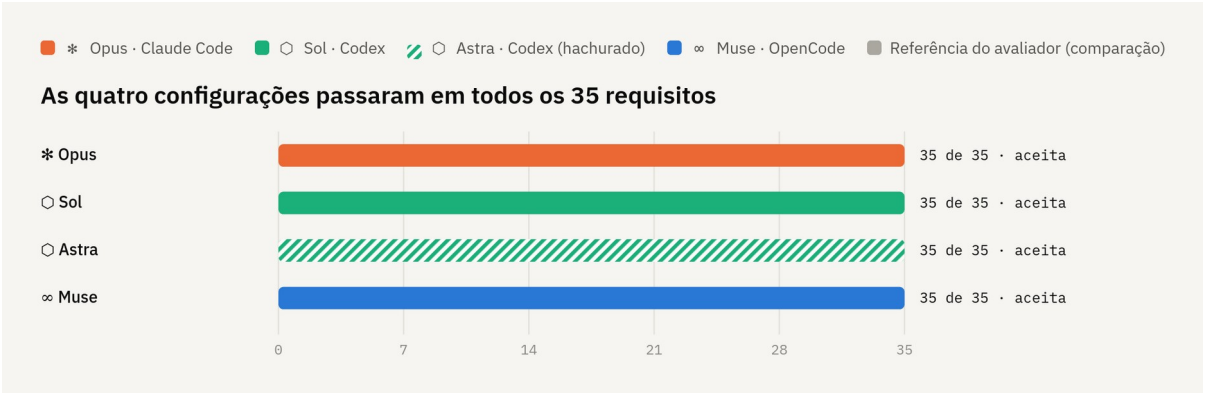


Figura 1. Requisitos satisfeitos por entrega aceita.

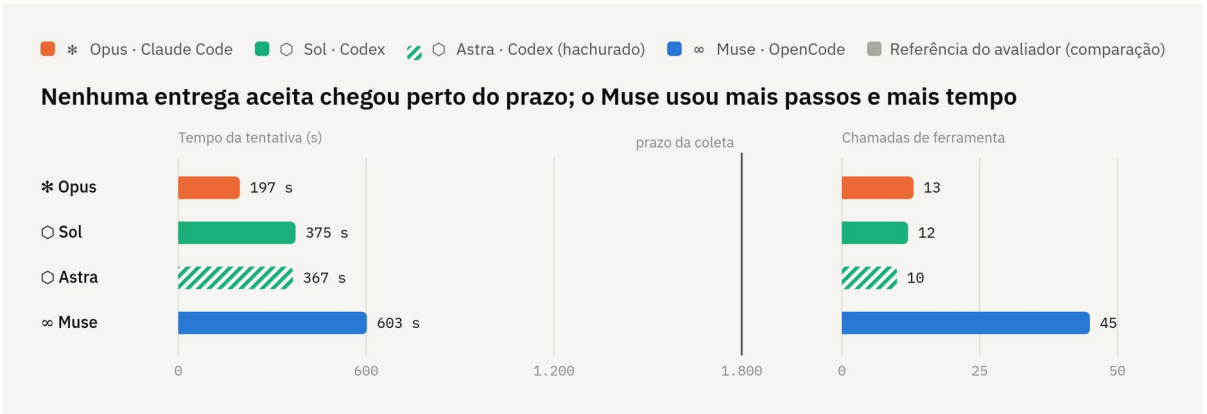


Figura 2. Tempo de trabalho do agente, com o prazo da coleta, e chamadas de ferramenta.

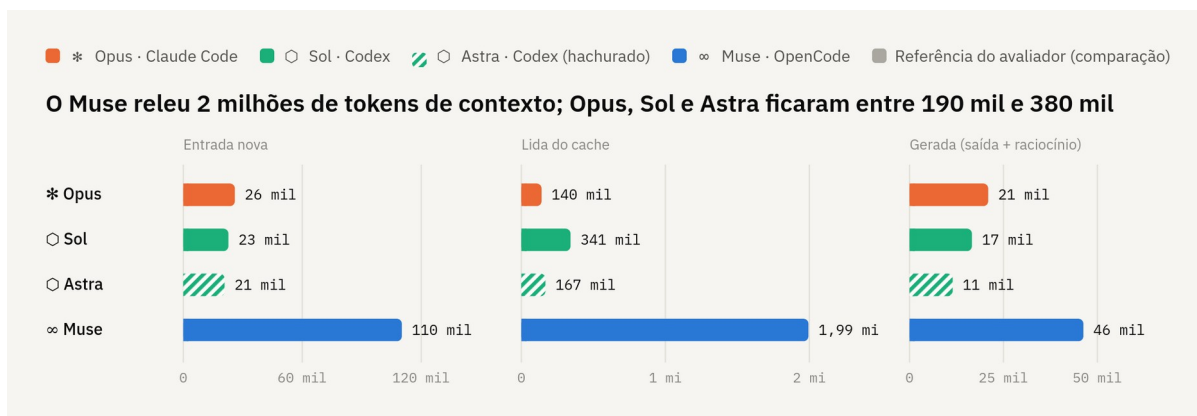


Figura 3. Tokens por tentativa, normalizados entre harnesses; cada painel tem sua escala.

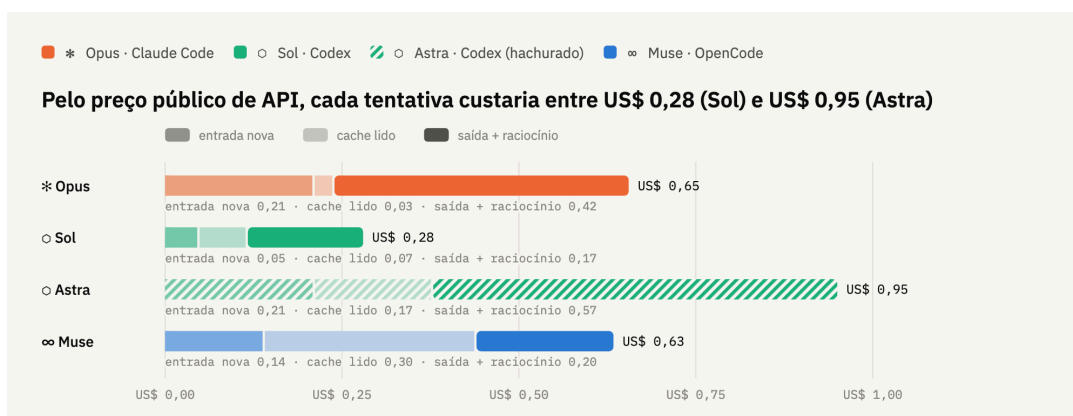


Figura 4. Custo estimado pelo preço público de API (não é o valor pago).

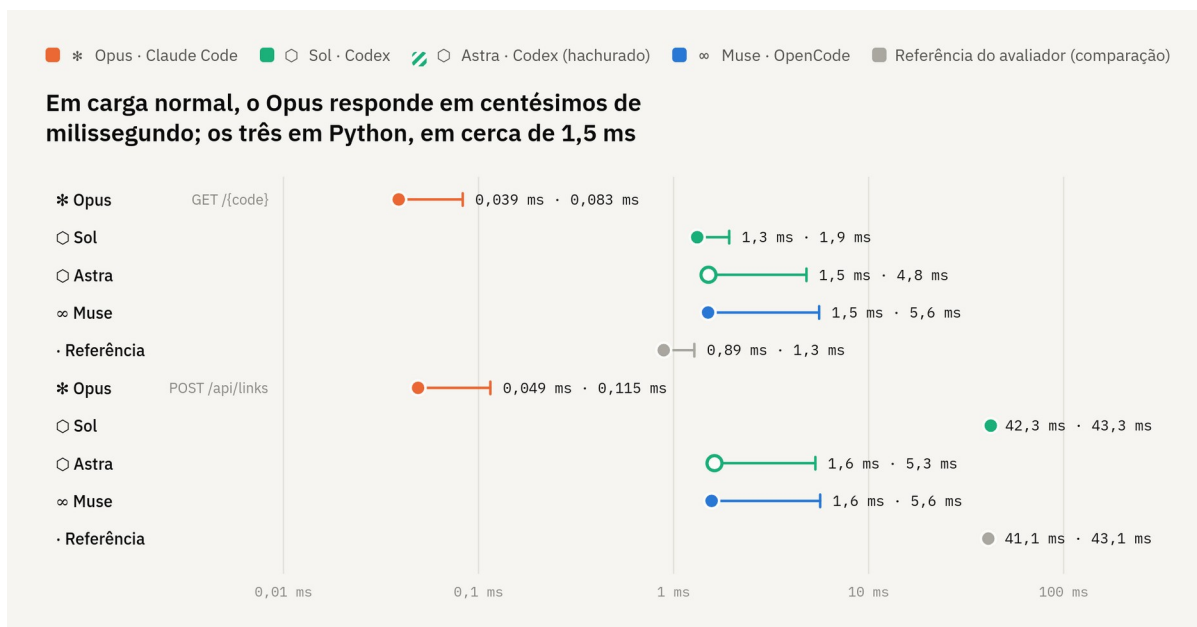


Figura 5. Latência em carga normal (500 req/s, dados em disco): ponto = p50, ponta = p99. Escala logarítmica; a referência do avaliador em cinza.

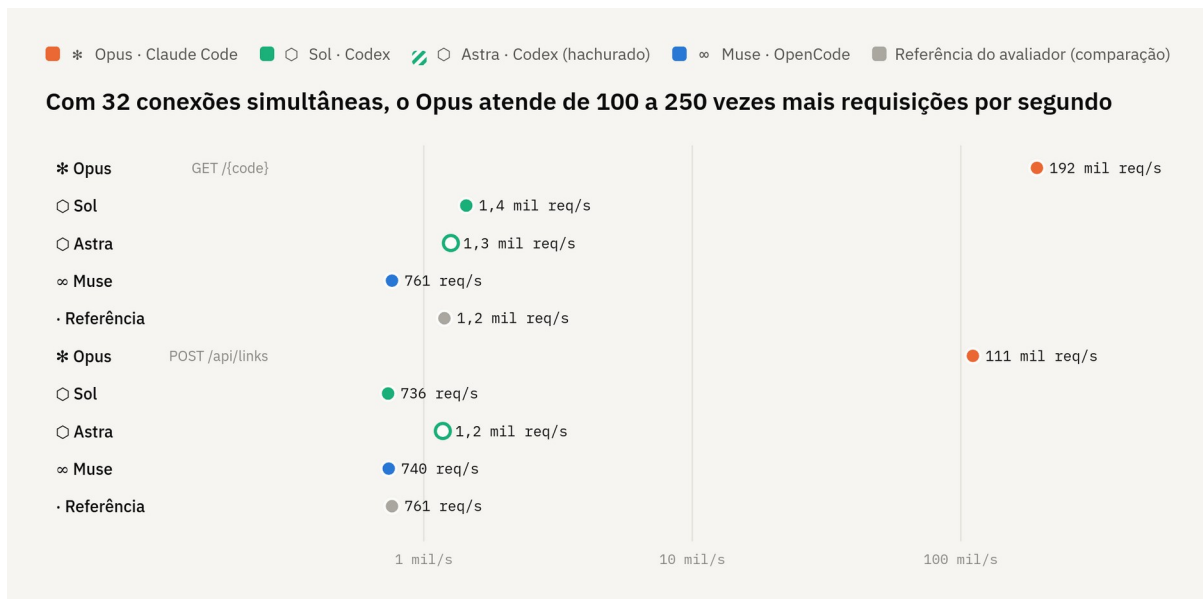


Figura 6. Vazão com 32 conexões simultâneas. O Opus atingiu o teto do teste; o valor é um limite inferior.

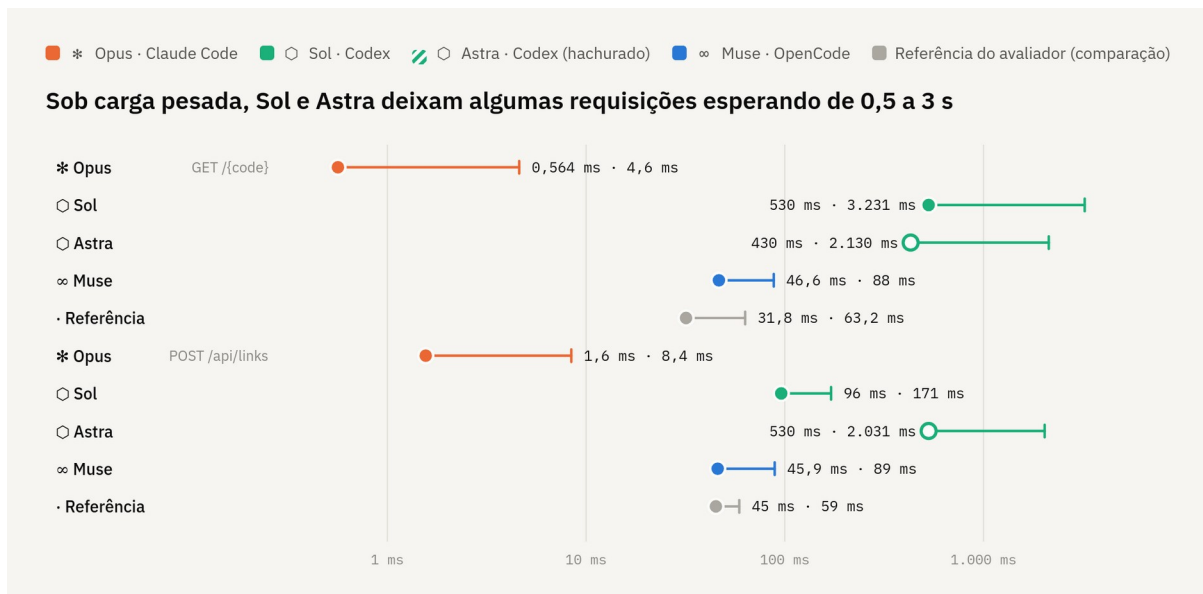


Figura 7. Sob saturação: ponto = p99, ponta = pior caso. Escala logarítmica.

Leitura dos resultados

- **Aceitação não distingue as configurações:** como esperado para modelos de fronteira, todos passaram. A comparação fica em tempo, tokens, custo, stack, desempenho e, na coleta, na revisão qualitativa.
- **Processo:** o Opus foi o mais rápido e com menos releitura de contexto; o Muse fez 45 passos e releu quase 2 milhões de tokens; o Astra gerou menos tokens, mas o preço por token o torna o mais caro.
- **Stack:** o Go do Opus responde em centésimos de milissegundo; parte da vantagem vem de não forçar gravação em disco a cada escrita (atende ao contrato, que só exige sobreviver a SIGTERM), enquanto as entregas em Python gravam com fsync a cada operação.

- **Implementação pesa tanto quanto a stack:** Sol, Astra e Muse usam a mesma stack e diferem por decisões de HTTP. O Sol mantém a conexão aberta e envia cabeçalhos e corpo separados, o que trava o POST em 41 ms pela interação com a confirmação atrasada do TCP; o Astra fecha a conexão a cada resposta e evita o atraso. Sob saturação, Sol e Astra têm picos de 0,4 a 3 s.

9. Problemas encontrados e corrigidos

O piloto existia para encontrar falhas do método antes da coleta. Os principais achados:

Problema	Efeito	Tratamento
Imagem sem o binário auxiliar do Codex	1ª tentativa do Sol sem ferramenta de execução; entrega vazia	Imagem reconstruída; tentativa repetida e aceita
Exportação de sessão do OpenCode truncada em 128 KiB	Modelo servido sem registro	Exportação por arquivo; verificada em tentativa real
Término por limite de saída invisível	Entrega vazia parecia execução normal	Runner registra como o harness terminou e emite avisos
Binários dos harnesses copiados do host	O Claude Code se atualizou sozinho entre builds	Binários fixados por hash; imagem exportada
Diferenças de paridade entre harnesses	OpenCode não usa /tmp; Codex recusa rm -rf	Documentadas, por decisão, sem corrigir

10. Limitações

- **Uma tentativa por configuração:** não há como medir a variação entre execuções, e uma diferença observada pode ser da execução, não da configuração.
- **Resultados do piloto não são oficiais:** o runner mudou durante o piloto, e parte das tentativas usou uma imagem anterior à correção.
- **Custo é estimativa:** aplica preços públicos de API aos tokens informados pelos harnesses, que contam de formas diferentes; foi feita uma normalização documentada.
- **Latência** mede o mesmo hardware, com cliente e servidor no mesmo host; não inclui rede real. A escrita em disco depende do SSD do host.
- **Qualidade interna (Q7)** ainda não foi avaliada: depende da rubrica qualitativa.
- **Isolamento** não foi certificado contra tentativas adversariais.

11. Conclusão

O piloto cumpriu seu papel: o método funciona de ponta a ponta, do envio do enunciado à avaliação automatizada em ambiente limpo e à medição de desempenho, e os quatro

participantes da coleta produziram entregas aceitas. As barreiras de isolamento se mantiveram em todas as tentativas, sem nenhum acesso indevido registrado.

Como a aceitação não distingue modelos de fronteira nessa tarefa, o valor do benchmark está nas medidas de processo e produto: o Opus foi o mais rápido e entregou a API de melhor desempenho; o Sol foi o mais barato; o Astra, o mais caro; o Muse, o que mais iterou. As diferenças de desempenho mostram que a stack importa, mas decisões de implementação dentro da mesma stack importam tanto quanto.

Os problemas mais importantes estavam no instrumento, não nos participantes, e foram corrigidos antes da coleta. **Próximos passos:** fixar os valores ainda indefinidos do contrato, definir a ordem das execuções, congelar os manifestos de configuração e a rubrica qualitativa, e executar a coleta oficial com uma tentativa por configuração.